UART-Controlled Multi-Clock System

(SYS_TOP)

Design, testbench, Lint and CDC Report

Andro Milad Ramzy

Contents

1. Introduction

2. System Specification

3. Architecture

4. Block Descriptions

5. Functional Verification

6. Lint Results

7. Clock Domain Crossing (CDC) Results

8. Conclusions and Recommendations

Appendix A: Complete Simulation Log

Appendix B: Reproducing the Results

1. Introduction

This report documents a complete, UART-controlled digital system implemented in Verilog. The system receives commands over a serial UART link, executes them on an 8-bit register file or a 16-function ALU, and returns the results over the same serial link through an asynchronous FIFO. It deliberately uses two unrelated clock domains and therefore includes the full set of clock-domain-crossing (CDC) structures: reset synchronizers, a data/enable synchronizer, a Gray-code asynchronous FIFO, programmable clock dividers and a clock gate.

The report covers the system specification, the architecture and block-level design, the functional verification (self-checking testbench, simulation log and waveform screenshots), and the results of the SpyGlass lint and CDC checks.

Project deliverables used for this report:

- RTL/ - all design sources (top level SYS_TOP.v plus UART, FIFO, clocking, synchronizer, ALU, register-file and control blocks).
- Simulation/ - Final_System_tb.sv, filelist.f, run.do and wave.do for ModelSim.
- Lint/ - SpyGlass project file, waiver files and summary reports for the lint and CDC runs.
- Final_system_report.pdf - waveform screenshots and the original simulation log.

2. System Specification

2.1 Key features

- Serial command interface: 115 200 baud, 8 data bits, even parity (configurable), one stop bit, LSB first.
- Four commands: register-file write, register-file read, ALU operation with operands, ALU operation on the stored operands.
- 16-entry x 8-bit register file; four registers have a dedicated function (ALU operands, UART configuration, TX clock divider).
- 16-function ALU (arithmetic, logic, comparison, shift) with a 16-bit result, clocked through a gated clock.
- Two clock domains (REF_CLK and UART_CLK) bridged by a 16-deep, 8-bit asynchronous FIFO with Gray-coded pointers.
- Receiver with 32x oversampling and 3-sample majority voting, parity and stop-bit error flags.

2.2 Interface and clocking

Signal / clock	Direction	Description
REF_CLK	Input	System clock, 50 MHz (20 ns). Clocks SYS_CTRL, RegFile, ALU (gated), DATA_SYNC and the FIFO write side.
UART_CLK	Input	UART reference clock, 3.6864 MHz (271.267 ns). Source of RX_CLK and TX_CLK.
RST	Input	Asynchronous, active-low reset. Synchronized separately into each clock domain (RST_SYNC).
RX_IN	Input	UART serial receive line.
TX_OUT	Output	UART serial transmit line.
TX_OUT_V	Output	High while the transmitter is busy sending a frame.
RX_Parity_Error	Output	Parity error flag from UART_RX.
RX_Stop_Error	Output	Stop-bit error flag from UART_RX.

Signal / clock	Direction	Description
RX_CLK	Internal	UART_CLK divided according to the prescale field (prescale 32 gives a divide ratio of 1, i.e. 3.6864 MHz = 32 x baud rate).
TX_CLK	Internal	UART_CLK / REG3 (reset value 32), i.e. 115.2 kHz = one bit per clock.

Parameters of SYS_TOP: DATA_WIDTH = 8, REG_FILE_DEPTH = 16, FIFO_DEPTH = 16, PRESCALE_WIDTH = 6, RST_SYNC_STAGES = 2, DATA_SYNC_STAGES = 2.

2.3 UART frame

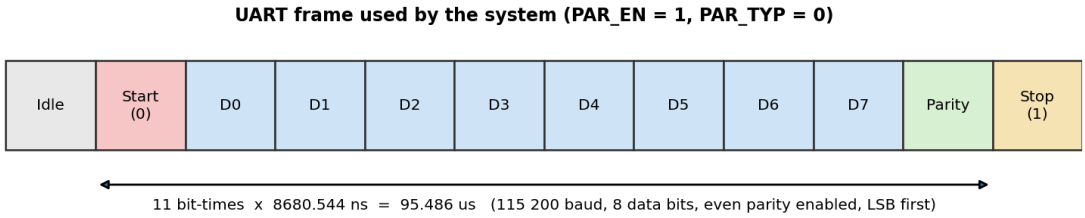


Figure 1 - UART frame format (start bit, 8 data bits LSB first, even parity, stop bit).

2.4 Command protocol

Every command is a sequence of UART frames. The first byte selects the command and SYS_CTRL decodes the remaining bytes in order. ALU results are returned as two bytes, least-significant byte first.

Command byte	Name	Frames sent to the system	Response
`0xAA`	Register write	[0xAA] [address] [data]	None
`0xBB`	Register read	[0xBB] [address]	1 byte (register value)
`0xCC`	ALU with operands	[0xCC] [operand A] [operand B] [ALU_FUN]	2 bytes (result LSB, MSB)
`0xDD`	ALU without operands	[0xDD] [ALU_FUN]	2 bytes (result LSB, MSB)

For command 0xCC the two operands are first written into REG0 and REG1 of the register file; command 0xDD reuses the values already stored there.

2.5 Register file map

Register	Reset value	Function
REG0	0x00	ALU operand A
REG1	0x00	ALU operand B
REG2	0x81	UART configuration: bit 0 = PAR_EN, bit 1 = PAR_TYP (0 = even), bits 7:2 = RX prescale (0b100000 = 32)
REG3	0x20	TX clock divider ratio (32 gives 115 200 baud from UART_CLK)
REG4 - REG15	0x00	General-purpose storage

2.6 ALU functions

ALU_FUN	Operation	Result	ALU_FUN	Operation	Result
0x0	Add	A + B	0x8	XOR	A ^ B
0x1	Subtract	A - B	0x9	XNOR	~(A ^ B)

ALU_FUN	Operation	Result	ALU_FUN	Operation	Result
0x2	Multiply	A x B	0xA	Equal	1 if A == B else 0
0x3	Divide	A / B	0xB	Greater than	2 if A > B else 0
0x4	AND	A & B	0xC	Smaller than	3 if A < B else 0
0x5	OR	A | B	0xD	Shift right	A >> 1
0x6	NAND	~(A & B)	0xE	Shift left	A << 1
0x7	NOR	~(A | B)	0xF	No operation	0

3. Architecture

Figure 2 shows the top-level structure. The design is split into a REF_CLK domain (command processing, register file, ALU, FIFO write side) and a UART_CLK domain (serial receiver and transmitter, clock dividers, FIFO read side). Three kinds of signal cross between the domains: the received byte and its valid strobe (UART to SYS_CTRL), the Gray-coded FIFO pointers, and the FIFO memory contents.

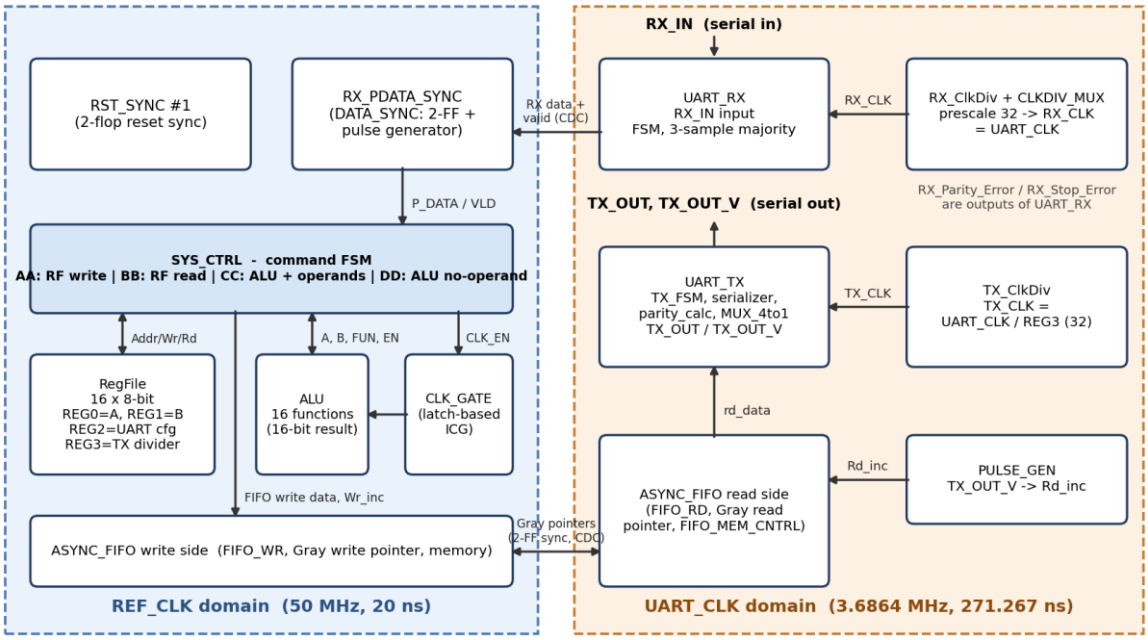


Figure 2 - Simplified block diagram of SYS_TOP showing both clock domains and the crossings between them.

Configuration values stored in the register file (REG2 and REG3) are static during operation and are used directly by the UART blocks and the TX clock divider.

3.1 Module hierarchy

Module	Source file	Role
SYS_TOP	Top/SYS_TOP.v	Top level; instantiates and connects all blocks.
SYS_CTRL	SYS_CTRL/SYS_CTRL.v	Command decoder FSM (11 states); drives register file, ALU and FIFO.
RegFile	RegFile/RegFile.v	16 x 8-bit register file with four dedicated output registers.
ALU	ALU/ALU.v	Synchronous 16-function ALU with status flags.
CLK_GATE	Clock_Gating/CLK_GATE.v	Latch-based clock gate for the ALU clock.

Module	Source file	Role
UART_TOP / UART_TX / UART_RX	UART/...	Serial transmitter and receiver (see 4.2).
ASYNC_FIFO	ASYNC_FIFO/Async_FIFO.v	Dual-clock FIFO: FIFO_WR, FIFO_RD, FIFO_MEM_CNTRL, DF_SYNC, bin2gray, gray2bin.
DATA_SYNC	DATA_SYNC/*.v	Multi-flop enable synchronizer, pulse generator and bus capture register.
RST_SYNC	RST_SYNC/RST_SYNC.v	Reset synchronizer (asynchronous assert, synchronous release).
ClkDiv / CLK_DIV_Counter	Clock_Divider/*.v	Programmable integer clock divider with bypass.
CLKDIV_MUX	CLKDIV_MUX/CLKDIV_MUX.v	Maps the prescale field (32/16/8/4) to a divide ratio (1/2/4/8).
PULSE_GEN	PULSE_GEN/PULSE_GEN.v	Turns the TX_OUT_V level into a one-cycle FIFO read pulse.

4. Block Descriptions

4.1 System controller and processing blocks

SYS_CTRL waits in IDLE for a valid received byte and decodes it as one of the four commands. For a register write it stores the address from the second frame and writes the data from the third frame. For a register read it issues a read and pushes the returned byte into the FIFO. For ALU commands it writes operands A and B into REG0 and REG1, applies the function code, waits for ALU_OUT_Valid and then pushes the result into the FIFO in two cycles (low byte first, then high byte), stalling while the FIFO is full. The ALU clock is enabled (through CLK_GATE) only for ALU commands.

The register file performs a write when WrEn is asserted and a read (with a one-cycle RdData_VLD strobe) when RdEn is asserted. The ALU is a clocked block with a case statement over the 16 function codes; it produces a 16-bit result, a valid strobe and arithmetic/logic/compare/shift/carry flags.

4.2 UART

The transmitter (UART_TX) consists of TX_FSM, a serializer, a parity calculator and a 4-to-1 output multiplexer that selects start bit, stop/idle level, serial data or parity bit. A frame is started whenever the FIFO is not empty, and the busy output (TX_OUT_V) is converted to a read pulse by PULSE_GEN.

The receiver (UART_RX) is built from an FSM, an edge/bit counter, a data sampler, a deserializer, and start, parity and stop checkers. The data sampler takes three samples around the middle of each bit (at edge counts prescale/2 - 1, prescale/2 and prescale/2 + 1) and uses a majority vote, which rejects single-sample glitches. A byte is flagged valid when the stop state completes without a parity or stop error.

4.3 Asynchronous FIFO

The FIFO has 16 entries of 8 bits (5-bit pointers). Each side keeps a binary pointer for addressing and a registered Gray-coded copy that is passed through a two-flop synchronizer (DF_SYNC) into the opposite domain, where it is converted back to binary for the full/empty comparison. Full is detected in the write domain when the pointers differ only in the MSB; empty

is detected in the read domain when the pointers are equal. The memory array is written with REF_CLK and read combinationally by the transmitter.

4.4 Clocking and reset

Two ClkDiv instances derive TX_CLK (divide ratio from REG3) and RX_CLK (divide ratio from CLKDIV_MUX) from UART_CLK; a ratio of 0 or 1 bypasses the divider. Two RST_SYNC instances (two flip-flops each) release the asynchronous reset synchronously in the REF_CLK and UART_CLK domains. CLK_GATE uses an active-low transparent latch followed by an AND gate so the ALU clock can be switched off without glitches. DATA_SYNC passes the received byte across domains: the valid strobe goes through a multi-flop synchronizer and a pulse generator, and the byte is captured in REF_CLK on that pulse.

5. Functional Verification

5.1 Verification environment

Verification uses a self-checking SystemVerilog testbench (Final_System_tb) that instantiates SYS_TOP and acts as the UART host. The testbench generates REF_CLK (20 ns) and UART_CLK (271.267 ns), applies reset, and drives and monitors the serial lines at 115 200 baud (bit period 8680.544 ns, frame period 95.486 us). Reusable tasks build and check the traffic:

- send_frame / drive_tx - builds an 11-bit frame (start, 8 data bits, even parity, stop) and shifts it onto RX_IN.
- recive_frame - waits for the falling edge of TX_OUT, samples in the middle of each bit and rebuilds the received frame.
- Test_RF_Wr / Test_RF_Rd - register write, and register read with the result compared against the register array inside the DUT.
- Test_ALU_OP / Test_ALU_NO_OP - issue an ALU command, read the two result bytes, compare with a reference model written in the testbench and update the passed/failed counters.

The simulation is started in ModelSim with run.do (vlib, vlog -sv with filelist.f, vsim, wave.do, run -all). The screenshots in section 5.4 were captured from ModelSim. The log in section 5.3 was produced by re-running the same testbench for this report (see the note below).

> **Note on the simulator used for the log.** ModelSim was not available for the log run, so the unmodified RTL was simulated with Icarus Verilog 12.0. Icarus cannot parse default values on task ports, so a simulation-only copy of the testbench with those defaults removed was used (every call passes all arguments, so behaviour is unchanged; the original file is untouched). Data values are identical to the ModelSim log in Final_system_report.pdf; event timestamps after the first test are shifted by exactly one UART_CLK period (271 ns).

5.2 Test cases

The testbench runs 17 checked transactions and one unchecked write, covering every command type and every ALU function code once.

#	Test	Stimulus	Checked against
1	Register write	Write 0xFF to address 0x05	Verified by test 2 (read-back)
2	Register read	Read address 0x05	Register array in the DUT
3 - 6	Arithmetic	ADD 5,3 / SUB 9,4 / MUL 2,5 / DIV 15,3	Reference model in testbench

#	Test	Stimulus	Checked against
7 - 12	Logic	AND 5,3 / OR 9,4 / NAND 2,5 / NOR 15,3 / XOR 2,5 / XNOR 15,3	Reference model in testbench
13 - 15	Comparison	Equal 5,3 / function 0xB 9,4 / function 0xC 9,4	Reference model in testbench
16 - 17	Shift	Right shift 5 / left shift 9	Reference model in testbench
18	ALU no-operand	Function 0xF (NOP) using stored operands	Expected result 0

Tests 2 to 18 are the 17 pass/fail checks counted by the testbench; the register write is verified indirectly by the read-back.

5.3 Simulation results and log

Summary: passed tests: 17, failed tests: 0. All 67 frames sent to the design were accepted by the receiver (the valid strobe from UART_RX pulsed 67 times), and the simulation finished at 10.283 ms of simulated time.

Test	A	B	ALU_FUN	Expected	Received	Result
Register read (addr 0x05)	–	–	–	0xFF	0xFF	PASS
ADD	0x05	0x03	0x00	0x0008	0x0008	PASS
SUB	0x09	0x04	0x01	0x0005	0x0005	PASS
MUL	0x02	0x05	0x02	0x000A	0x000A	PASS
DIV	0x0F	0x03	0x03	0x0005	0x0005	PASS
AND	0x05	0x03	0x04	0x0001	0x0001	PASS
OR	0x09	0x04	0x05	0x000D	0x000D	PASS
NAND	0x02	0x05	0x06	0xFFFF	0xFFFF	PASS
NOR	0x0F	0x03	0x07	0xFFF0	0xFFF0	PASS
XOR	0x02	0x05	0x08	0x0007	0x0007	PASS
XNOR	0x0F	0x03	0x09	0xFFF3	0xFFF3	PASS
Equal	0x05	0x03	0x0A	0x0000	0x0000	PASS
Greater than	0x09	0x04	0x0B	0x0002	0x0002	PASS
Smaller than	0x09	0x04	0x0C	0x0000	0x0000	PASS
Shift right	0x05	0x03	0x0D	0x0002	0x0002	PASS
Shift left	0x09	0x04	0x0E	0x0012	0x0012	PASS
ALU no-operand (NOP)	–	–	0x0F	0x0000	0x0000	PASS

Excerpt of the simulation log (first transactions; the complete log is in Appendix A):

```
Testing RF_Wr
Address:05,Data:ff
@time:96029000 frame0 sent
@time:191515000 frame1 sent
@time:287000000 frame2 sent
@time:287000000 Testing RF_Rd:
Address:05
@time:382486000 frame0 sent
@time:477972000 frame1 sent
data recived:ff
Read Command passed
@time:613201000 Testing ALU_OP:
op_A:05,op_B:03,ALU_FUN:00
@time:708687000 frame0 sent
@time:804173000 frame1 sent
@time:899659000 frame2 sent
@time:995145000 frame3 sent
```

```
data recived:0008
ALU Command passed
@time:1229522000 Testing ALU_OP:
op_A:09,op_B:04,ALU_FUN:01
@time:1325008000 frame0 sent
@time:1420494000 frame1 sent
@time:1515980000 frame2 sent
@time:1611466000 frame3 sent
data recived:0005
ALU Command passed
@time:1845843000 Testing ALU OP:
op A:02,op B:05,ALU FUN:02
@time:1941329000 frame0 sent
@time:2036815000 frame1 sent
```

End of log:

```
@time:10048986000 frame1 sent
data recived:0000
ALU Command passed
passed tests:          17
failed tests:           0
Final_System_tb_sim.sv:308: $finish called at 10283362610 (1ps)
```

5.4 Waveforms

The following ModelSim screenshots show the testbench-level signals (RX_IN and TX_OUT serial lines, error flags, TX_OUT_V, PAR_EN/PAR_TYP, RX prescale and the frame captured by the testbench). The configuration is PAR_EN = 1, PAR_TYP = 0 and RX prescale = 100000 (32). In every capture recived_frame holds the 11-bit frame as {stop, parity, data[7:0], start}.

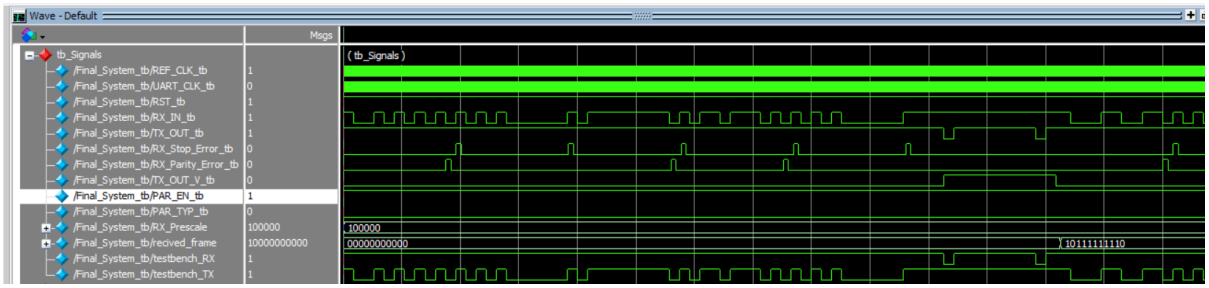


Figure 3 - Start of the simulation: register write/read traffic. The captured response frame 10111111110 is start = 0, data = 0xFF, parity = 0 (even), stop = 1, i.e. the value 0xFF written to address 0x05.

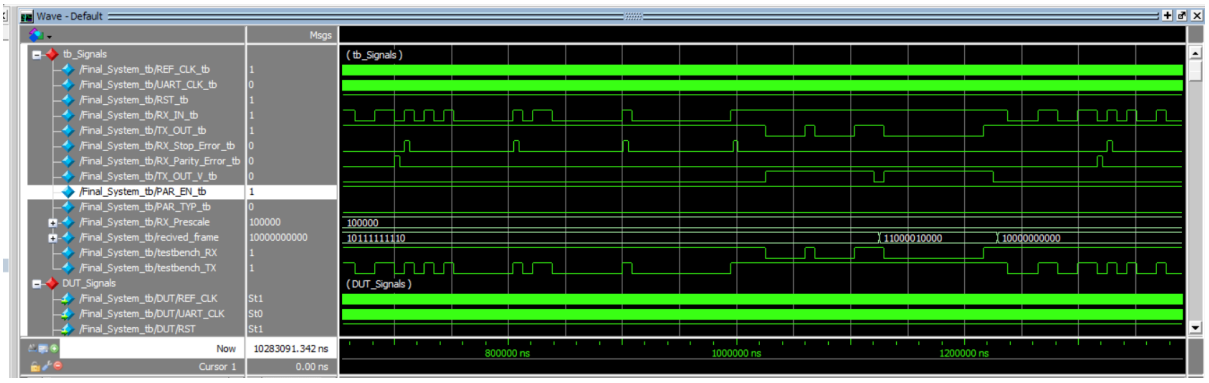


Figure 4 - ADD (5 + 3) response. The captured frame 11000010000 carries data 0x08 with parity 1, followed by the idle value for the high byte (0x00).

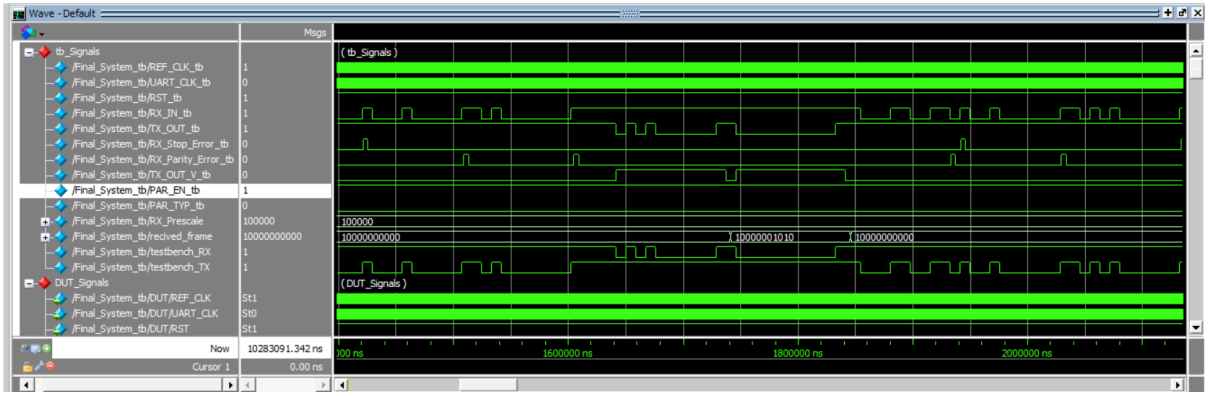


Figure 5 - SUB (9 - 4) response. The captured frame 10000001010 carries data 0x05 with parity 0.

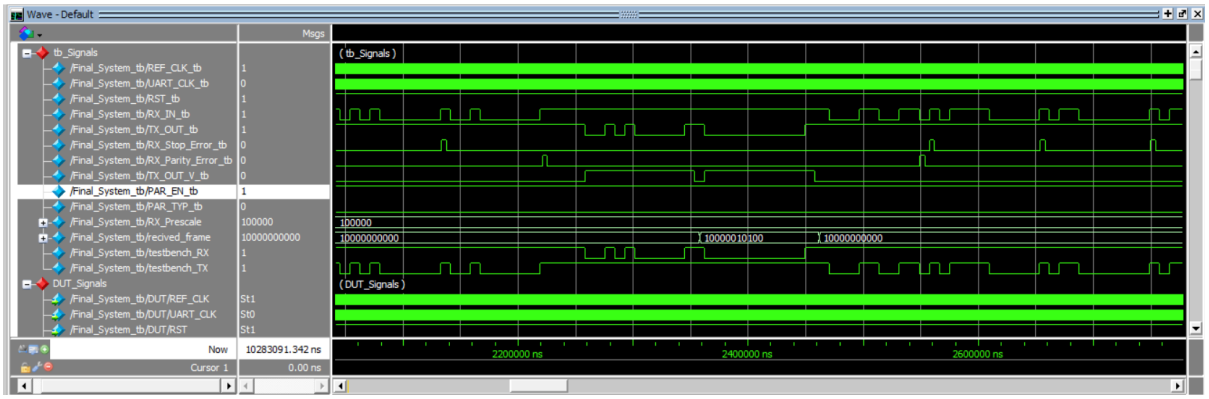


Figure 6 - MUL (2 x 5) response. The captured frame 10000010100 carries data 0x0A with parity 0.

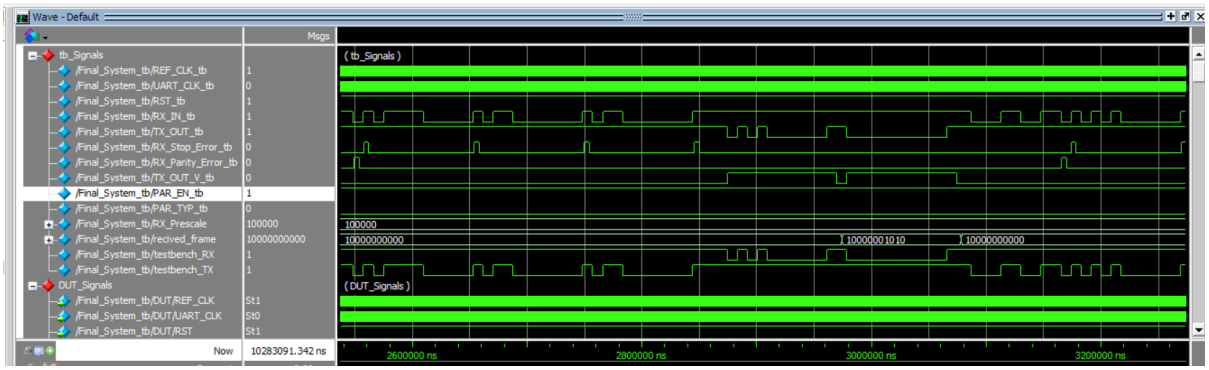


Figure 7 - DIV (15 / 3) response. The captured frame 10000001010 carries data 0x05 with parity 0.

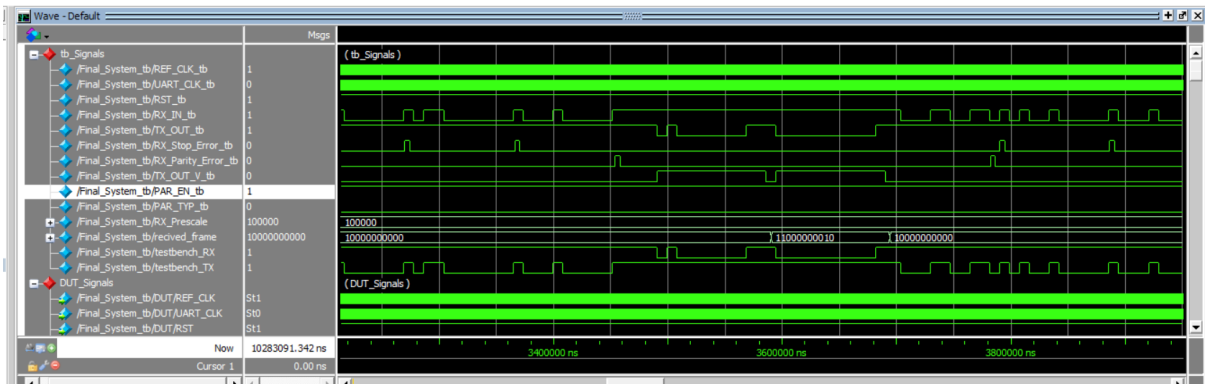


Figure 8 - AND (5 & 3) response. The captured frame 11000000010 carries data 0x01 with parity 1.

In each case TX_OUT_V rises for the duration of a transmitted frame, the parity bit equals the even parity of the data byte, and the byte reconstructed by the testbench matches the log values in section 5.3.

5.5 Observations from the simulation

Error-flag pulses on valid frames. The waveforms show short pulses on RX_Stop_Error and RX_Parity_Error while perfectly valid frames are being received. Instrumenting the run counted 51 rising edges on RX_Stop_Error and 35 on RX_Parity_Error (the counts may include a start-up transition at time 0) against 67 accepted frames, each pulse about 4.34 us wide (half a bit time). No frame was rejected and no test failed, so the pulses do not affect the data path. A likely cause is that the stop and parity checks are enabled by the receiver state before the sampled bit of that field has settled, and the flags are exported directly. If these two outputs are used by a host, they should be qualified (for example registered and sampled only at the end of the frame).

Function-code naming in the testbench. The ALU implements 0xB as 'greater than' (result 2) and 0xC as 'smaller than' (result 3), but the testbench constants SML = 0x0B and GRT = 0x0C use the opposite names. The expected values are computed from the function code, so the checks are correct; only the constant names are swapped.

Coverage of the test plan. The test plan exercises each command and each function once with one operand pair. It does not yet cover error injection (bad parity or stop bit), odd parity, other prescale values, FIFO-full back-pressure, back-to-back commands, reset in the middle of a transfer, or division by zero.

6. Lint Results

Lint was run with SpyGlass (SpyGlass_vL-2016.06) on the whole design with SYS_TOP as the top-level unit, using the lint/lint_rtl goal of the rtl_handoff methodology. The enabled policies were lint, morelint, openmore, starc, starc2005, erc, latch, simulation and timing.

Metric	Value
Top design unit	SYS_TOP
Total messages generated	10
Messages waived	7
Messages reported (remaining)	3, all severity Info (top-unit detection, elaboration summary, and a pointer to the signal-usage report)
Remaining errors or warnings	0

All seven waived messages are intentional or harmless and are documented in the waiver file:

Rule	Severity	Location	Message	Reason for waiving
W240	Warning	UART_RX/FSM.v:9	Input edge_cnt[4:0] declared but not read	The receive FSM takes its timing from bit_cnt and bit_done; the port is unused. It can be removed.
W528	Warning	SYS_TOP.v:179	Shift_flag set but not read	ALU status flag not needed by the controller.
W528	Warning	SYS_TOP.v:180	CMP_flag set but not read	Same as above.
W528	Warning	SYS_TOP.v:181	Logic_flag set but not read	Same as above.
W528	Warning	SYS_TOP.v:182	Arith_flag set but not read	Same as above.
W528	Warning	SYS_TOP.v:183	Carry_Flag set but not read	Same as above.
InferLatch	Error	CLK_GATE.v:22	Latch inferred for Latch_Out	Intentional: the clock gate needs a transparent-low latch. The source notes a library ICG cell (TLATNCAX12M) for synthesis.

Result: the RTL is lint-clean once these seven items are waived. The only item with Error severity is the deliberate clock-gate latch, and the remaining waivers are unused-signal warnings with no functional impact.

7. Clock Domain Crossing (CDC) Results

CDC analysis used the same SpyGlass version with the goals cdc/clock_reset_integrity, cdc/cdc_verify_struct and cdc/cdc_verify, all run on SYS_TOP with a constraint file (cdc_constraints.sgdc). The design has two asynchronous clock domains, REF_CLK and UART_CLK (RX_CLK and TX_CLK are divided versions of UART_CLK).

7.1 Crossings in the design

Crossing	From to	Structure
Received byte and valid	UART_RX (UART_CLK) to SYS_CTRL (REF_CLK)	DATA_SYNC: multi-flop synchronizer on the valid signal, pulse generator, bus captured on the pulse.
FIFO write pointer	REF_CLK to TX_CLK	Gray code, registered, two-flop synchronizer (DF_SYNC), then converted to binary.
FIFO read pointer	TX_CLK to REF_CLK	Same structure in the opposite direction.
FIFO data	FIFO memory (REF_CLK) to UART_TX serializer (TX_CLK)	Memory read directly; protected by the pointer handshake (data is stable before empty is released).
Reset	RST to both domains	RST_SYNC, two flops per domain.

7.2 Reported items and their disposition

The CDC waiver file contains seven waivers. They fall into three rule families, all from the structural CDC check:

Rule	Count	Reported situation	Disposition
Ac_conv04	3	Bits of FIFO memory (62/23, 54/15, 78/39) reach UART_TX serializer DATA through a different synchronization scheme than the other bits.	Waived. The memory is a standard async-FIFO array that is read only after the synchronized Gray pointers indicate it is non-empty, so the data is stable when sampled.
Ac_conv02	3	Synchronized signals converge on combinational logic: 5 bits of the write-pointer synchronizer (bin_wq2_rptr[0]), 5 bits of the read-pointer synchronizer (bin_rq2_wptr[0]), and 9 flops of RX_PDATA_SYNC at SYS_CTRL.next_state[2]. The Gray-coding check is disabled.	Waived. The pointer convergence is the Gray-to-binary XOR network after the synchronizers, which is the accepted FIFO structure. The RX data bus is captured only on the enable pulse, so it is stable when SYS_CTRL uses it.
Ac_conv01	1	14 synchronizers (FIFO pointer and RX data/valid) converge on the combinational output Ref_File_WrEn.	Waived. The two crossings are independent protocols that only meet in the REF_CLK controller logic and need no mutual coherence.

The three Ac_conv04 waivers were created in separate sessions, each for a different pair of bits of the same memory-to-serializer path, so they represent a single structural situation reported three times.

Scope of this section. The files provided for the CDC runs are the project file and the waiver file. The complete CDC violation report and the SGDC constraint file are not part of the archive, so this section lists the waived items and the setup, and the reasons in the Disposition column are derived from the RTL structure. They should be checked against the full SpyGlass CDC report.

Result: every CDC message recorded in the project is a convergence or memory-array item on a standard synchronizer structure; none of the recorded items indicates a missing synchronizer on a single-bit control signal.

8. Conclusions and Recommendations

The UART-controlled system works as specified. The self-checking testbench passes all 17 checks (register read-back and all 16 ALU function codes, including the no-operand command), the 67 transmitted frames are all accepted, and the responses seen on TX_OUT in the waveforms carry the correct data and even parity. SpyGlass lint is clean apart from seven waived items, one of which is the intentional clock-gating latch, and the CDC structure consists of standard synchronizers (reset synchronizers, enable-pulse data synchronizer and Gray-code FIFO pointers).

Recommended follow-up work:

- Qualify the RX_Parity_Error and RX_Stop_Error outputs (register them and sample only at the end of the frame) so they do not pulse on valid frames.
- Extend the testbench with negative tests (bad parity, bad stop bit), odd parity, other prescale values (16, 8, 4), FIFO-full back-pressure, back-to-back commands, reset during a transfer and division by zero; add code and functional coverage.
- Correct the swapped SML/GRT constant names in the testbench, and compare register reads against an independent reference rather than the DUT's own array.
- Remove the unused edge_cnt input of the receive FSM, and either use or remove the unused ALU flag outputs at the top level to reduce waivers.
- Replace the behavioural clock-gate latch with the library ICG cell for synthesis, as already noted in CLK_GATE.v.
- Fix the filelist.f entry Pulse_gen.v to Pulse_Gen.v so the flow also works on case-sensitive file systems.
- Archive the complete SpyGlass CDC report and constraint file with the project.

Appendix A: Complete Simulation Log

Complete output of the testbench (blank lines removed). Times are in picoseconds.

```
Testing RF_Wr
Address:05,Data:ff
@time:96029000 frame0 sent
@time:191515000 frame1 sent
@time:287000000 frame2 sent
@time:287000000 Testing RF_Rd:
Address:05
@time:382486000 frame0 sent
@time:477972000 frame1 sent
data recived:ff
Read Command passed
@time:613201000 Testing ALU_OP:
op_A:05,op_B:03,ALU_FUN:00
@time:708687000 frame0 sent
@time:804173000 frame1 sent
@time:899659000 frame2 sent
@time:995145000 frame3 sent
data recived:0008
ALU Command passed
@time:1229522000 Testing ALU_OP:
op_A:09,op_B:04,ALU_FUN:01
@time:1325008000 frame0 sent
@time:1420494000 frame1 sent
@time:1515980000 frame2 sent
@time:1611466000 frame3 sent
data recived:0005
ALU Command passed
@time:1845843000 Testing ALU_OP:
op_A:02,op_B:05,ALU_FUN:02
@time:1941329000 frame0 sent
@time:2036815000 frame1 sent
@time:2132301000 frame2 sent
@time:2227787000 frame3 sent
data recived:000a
ALU Command passed
@time:2462164000 Testing ALU_OP:
op_A:0f,op_B:03,ALU_FUN:03
@time:2557650000 frame0 sent
@time:2653136000 frame1 sent
@time:2748622000 frame2 sent
@time:2844108000 frame3 sent
data recived:0005
ALU Command passed
@time:3078485000 Testing ALU_OP:
op_A:05,op_B:03,ALU_FUN:04
@time:3173971000 frame0 sent
@time:3269456000 frame1 sent
@time:3364942000 frame2 sent
@time:3460428000 frame3 sent
data recived:0001
ALU Command passed
@time:3694805000 Testing ALU_OP:
op_A:09,op_B:04,ALU_FUN:05
@time:3790291000 frame0 sent
@time:3885777000 frame1 sent
@time:3981263000 frame2 sent
@time:4076749000 frame3 sent
data recived:000d
ALU Command passed
@time:4311126000 Testing ALU_OP:
op_A:02,op_B:05,ALU_FUN:06
@time:4406612000 frame0 sent
@time:4502098000 frame1 sent
@time:4597584000 frame2 sent
@time:4693070000 frame3 sent
data recived:ffff
ALU Command passed
@time:4927447000 Testing ALU_OP:
op_A:0f,op_B:03,ALU_FUN:07
@time:5022933000 frame0 sent
@time:5118419000 frame1 sent
@time:5213905000 frame2 sent
@time:5309391000 frame3 sent
data recived:fff0
ALU Command passed
@time:5543768000 Testing ALU_OP:
op_A:02,op_B:05,ALU_FUN:08
@time:5639254000 frame0 sent
```

```
@time:5734740000 frame1 sent
@time:5830226000 frame2 sent
@time:5925712000 frame3 sent
data recived:0007
ALU Command passed
@time:6160089000 Testing ALU_OP:
op_A:0f,op_B:03,ALU_FUN:09
@time:6255575000 frame0 sent
@time:6351061000 frame1 sent
@time:6446547000 frame2 sent
@time:6542033000 frame3 sent
data recived:fff3
ALU Command passed
@time:6776410000 Testing ALU_OP:
op_A:05,op_B:03,ALU_FUN:0a
@time:6871896000 frame0 sent
@time:6967382000 frame1 sent
@time:7062868000 frame2 sent
@time:7158354000 frame3 sent
data recived:0000
ALU Command passed
@time:7392731000 Testing ALU_OP:
op_A:09,op_B:04,ALU_FUN:0b
@time:7488217000 frame0 sent
@time:7583703000 frame1 sent
@time:7679189000 frame2 sent
@time:7774675000 frame3 sent
data recived:0002
ALU Command passed
@time:8009052000 Testing ALU_OP:
op_A:09,op_B:04,ALU_FUN:0c
@time:8104538000 frame0 sent
@time:8200024000 frame1 sent
@time:8295510000 frame2 sent
@time:8390996000 frame3 sent
data recived:0000
ALU Command passed
@time:8625373000 Testing ALU_OP:
op_A:05,op_B:03,ALU_FUN:0d
@time:8720859000 frame0 sent
@time:8816345000 frame1 sent
@time:8911831000 frame2 sent
@time:9007317000 frame3 sent
data recived:0002
ALU Command passed
@time:9241693000 Testing ALU_OP:
op_A:09,op_B:04,ALU_FUN:0e
@time:9337179000 frame0 sent
@time:9432665000 frame1 sent
@time:9528151000 frame2 sent
@time:9623637000 frame3 sent
data recived:0012
ALU Command passed
@time:9858014000 Testing ALU_NO_OP
ALU_FUN:0f
@time:9953500000 frame0 sent
@time:10048986000 frame1 sent
data recived:0000
ALU Command passed
passed tests:         17
failed tests:          0
Final_System_tb_sim.sv:308: $finish called at 10283362610 (1ps)
```

Appendix B: Reproducing the Results

ModelSim

From the Simulation directory:

```
vsim -c -do run.do    (batch)  or  do run.do in the GUI to also load wave.do
```

Icarus Verilog (used for the log in this report)

```
iverilog -g2012 -s Final_System_tb -o sim.vvp -f filelist_no_tb.f Final_System_tb_sim.sv
vvp sim.vvp > sim_log.txt
```

filelist_no_tb.f is filelist.f without the testbench line and with the case of Pulse_Gen.v corrected; Final_System_tb_sim.sv is the testbench with default values removed from the task port declarations.